Linear Data Structures Continued :→ {Principle} {Last In First Out}

The Stack Data Structure :→ (Representation Using Array)

* One simple variable to represent everything :

  int top = -1; { Represents an empty stack }

* One variable to control the capacity / size (MAX_SIZE)

arr = | x | | | | | | |

(index) ← { top = -1 } 0

(push) → edge case (FULL) × top == -1 ← is Empty ()

(pop) → edge case (EMPTY) × top == MS - 1 is Full ()

boolean isFull () {
  if (top == MS - 1) {
    → true;
  }
}

a	b	c	d	e
0	1	2	3	4

(top)

Max Size = 5

5 - 1 = 4

| S |

push () → top == MS - 1

(Stack Overflow) message

Stack | X |

top = -1 0

pop () → top == -1

message

(Stack Underflow)

3
2
1

boolean is Empty () {
  if (top == -1)
    → true;
}

* { (arr) (top | MAX_SIZE) }

peek → top element

display ();

public int size () {
  return top + 1;
}

s = 0 -1 + 1 = 0

-1 empty top = -1

s = s + 1 = 6

a	y	z	a	b	c
0	1	2	3	4	5

top = 5

s = 1 + 1 = 2

x	y
0	1

top = 1

Max-size = 10

1	2	3	4
0	1	2	3

top - -

4

* Use cases of Stack (Last In First Out) (Not In Sequence) ≡ (Reverse)

* Palindrome → no 1 2 2 1 → stack null → stack

LL 1 → 2 → 1 →

1 2 2 1

* Valid Parentheses or Balanced Parentheses :→

String str1 = " { ([( )]) } ";

String str2 = " { ([ )] } ";

Intuition: (Pseudo Code) :→

if str[i] = (or { or [
  push to stack.
else
  if st.empty()
    → (false)
char ch = s.top()

~~[~~	[
~~(~~	(
~~{~~	{

if s[i] ==)]
  && c != (
  return true;
) == c [
  return false;

* Stack Based Expression Evaluation :→ (IMP)

{ First Let's understand Postfix Expression } (Postfix)

Normal Mathematics (Infix) 2 + 3 * 4

Postfix Format: 2 3 4 + *

Rules of postfix:

1. Numbers come first
2. Operators come after operands (numbers)

eg: 23 * 54 * + 9 - ⇒ (2 * 3) + (5 * 4) - 9

Algorithm :→
1. If number → push to stack
2. If operator → pop 2 numbers
3. Perform the operation
4. Push the result back to stack

Expression :→ 23 * 54 * + 9 -

Step	Symbol	Stack	Operation
1	2	2	push
2	3	2 3	push
3	*	6	2×3
4	5	6 5	push
5	4	6 5 4	push
6	*	6 20	5×4
7	+	26	6+20
8	9	26 9	push
9	-	17	26 - 9

Final Answer is 17

c → digit

2 ch → digit

+ → not digit

int c - '0'